

Resumen del video y funciones de C++ usadas para crear el videojuego

Video: "Make a C++ game FULL COURSE!" - Low Level Game Dev

Este documento no se basa en una transcripción automática parcial, sino en el repositorio oficial enlazado por el autor del video. Por eso resume las funciones y piezas de código que efectivamente aparecen en la implementación del juego.

Juego	Shooter espacial 2D con movimiento, disparos, enemigos, fondo con parallax y barra de vida.
Lenguaje	C++17
Librerías principales	GLFW, OpenGL/GLAD, GLM, gl2d, glui y raudio.
Base analizada	Repositorio oficial del curso enlazado en la descripción del video.

1. Resumen general del proyecto

El curso construye un juego 2D donde el jugador controla una nave espacial, se mueve con teclado, apunta con el mouse y dispara proyectiles. Los enemigos aparecen alrededor del jugador, lo persiguen con una lógica de giro progresivo y disparan cuando están alineados. El motor del ejemplo está armado con una capa de plataforma para ventana e input, una capa de juego con entidades y una capa de render/audio basada en librerías externas.

La arquitectura queda dividida, en lo esencial, en: **main y plataforma** (ventana, bucle principal, input, audio), **lógica del juego** (estado, spawn, colisiones, salud, reinicio), y **renderizado** (fondos, nave, balas, UI).

2. Funciones propias del juego

Inicialización y ciclo general

Función	Qué hace
initGame()	Inicializa recursos del juego: renderer, texturas, atlas, sonidos, fondos y cámara inicial.
gameLogic(float deltaTime)	Es la función central de cada frame: lee input, mueve al jugador, actualiza cámara, balas, enemigos, colisiones, UI y render final.
closeGame()	Punto de cierre del juego. En este proyecto queda vacío, pero existe para la limpieza futura.
restartGame()	Reinicia el estado principal del juego: jugador, enemigos, balas, salud y seguimiento de cámara.

Gameplay y reglas

Función	Qué hace
intersectBullet(glm::vec2 bulletPos, glm::vec2 shipPos, float shipSize)	Chequea colisión simple entre bala y nave usando distancia entre puntos.
spawnEnemy()	Crea un enemigo con posición alrededor del jugador y parámetros aleatorios de velocidad, giro, disparo y tipo.

Entidad bala

Función	Qué hace
Bullet::update(float deltaTime)	Avanza la bala según dirección, velocidad y tiempo transcurrido.
Bullet::render(...)	Dibuja la bala y su estela visual; cambia textura si la bala pertenece al enemigo.

Entidad enemigo

Función	Qué hace
Enemy::update(float deltaTime, glm::vec2 playerPos)	Gira hacia el jugador, avanza y decide si debe disparar.
Enemy::render(...)	Dibuja la nave enemiga usando la textura atlas y la orientación actual.

Render auxiliar

Función	Qué hace
TiledRenderer::render(gl2d::Renderer2D &renderer;)	Pinta el fondo repetido en mosaico para cubrir el área visible.
renderSpaceShip(...)	Renderiza una nave rotada según su vector de vista.

3. Funciones de entrada y plataforma

El proyecto no usa el input de forma desordenada dentro del gameplay. Primero define una pequeña API propia en el namespace **platform**, y luego esa API se alimenta con callbacks de GLFW. Esto simplifica mucho la lógica del juego.

Función	Uso en el proyecto
keyCallback(...)	Traduce eventos de teclado de GLFW a los botones internos del juego.
mouseCallback(...)	Actualiza el estado del mouse izquierdo y derecho.
windowFocusCallback(...)	Cuando la ventana pierde foco, resetea inputs para evitar teclas 'pegadas'.
cursorPositionCallback(...)	Marca si el mouse se movió en el frame.
characterCallback(...)	Acumula texto escrito por el usuario.
platform::getFramebufferSize()	Devuelve el tamaño real en píxeles del framebuffer.
platform::getWindowSize()	Devuelve el tamaño lógico de la ventana.
platform::getRelMousePosition()	Devuelve la posición del mouse relativa al área de dibujo.
platform::isFocused()	Indica si la ventana está enfocada.
platform::showMouse(bool)	Muestra u oculta el cursor.
platform::setRelMousePosition(int, int)	Reposiciona el cursor.
platform::writeEntireFile(...)	Escribe un archivo binario completo.
platform::readEntireFile(...)	Lee un archivo binario completo.
platform::isButtonHeld / PressedOn / Released / Typed	Consulta estados de teclas.
platform::isLMousePressed / Held / Released	Consulta estados del mouse izquierdo.
platform::getControllerButtons()	Lee el estado del gamepad si la ventana está enfocada.
platform::internal::updateAllButtons(float)	Actualiza todos los estados de botones y gamepad al final del frame.
platform::internal::resetInputsToZero()	Limpia entradas cuando hace falta.

4. Funciones y utilidades de C++/biblioteca estándar que aparecen en el código

Elemento	Dónde se usa
std::vector::push_back	Agregar balas y enemigos.
std::vector::erase	Eliminar balas lejanas, enemigos muertos o entidades fuera de rango.

Elemento	Dónde se usa
<code>std::vector::begin / size</code>	Recorrer y modificar vectores.
<code>std::srand(std::time(0))</code>	Inicializar aleatoriedad del juego.
<code>rand()</code>	Elegir tipo de enemigo, velocidad, rango de disparo y tiempos aleatorios.
<code>std::ofstream / std::ifstream</code>	Escritura y lectura completa de archivos.
<code>is_open(), write(), read(), close()</code>	Operaciones de archivo usadas en la capa de plataforma.
<code>std::chrono::high_resolution_clock::now()</code>	Tomar tiempos para calcular deltaTime.
<code>std::chrono::duration_cast</code>	Convertir diferencia temporal a segundos.

5. Funciones matemáticas y gráficas usadas constantemente

Aunque el juego está escrito en C++, gran parte de su lógica depende de funciones matemáticas de GLM y del renderizador gl2d. Estas son las más importantes porque explican cómo se logra movimiento, rotación, colisión y dibujo en pantalla.

Función/objeto	Finalidad
<code>glm::distance</code>	Detectar colisiones y medir distancia entre jugador, balas y enemigos.
<code>glm::normalize</code>	Obtener direcciones unitarias para movimiento y disparo.
<code>glm::length</code>	Verificar magnitud de vectores.
<code>glm::clamp</code>	Limitar vida y otros valores.
<code>glm::rotate</code> / <code>glm::radians</code> / <code>glm::degrees</code>	Rotar vectores o sprites y convertir ángulos.
<code>atan2</code>	Calcular orientación de nave y balas.
<code>glViewport</code>	Definir el área de render en OpenGL.
<code>glClear</code>	Limpiar la pantalla en cada frame.
<code>gl2d::Renderer2D::renderRectangle</code>	Dibujar fondos, naves, balas y barra de vida.
<code>gl2d::Renderer2D::flush</code>	Enviar el contenido renderizado al frame actual.
<code>renderer.currentCamera.follow</code>	Hacer que la cámara siga al jugador.
<code>renderer.pushCamera</code> / <code>popCamera</code>	Renderizar la UI sin afectar la cámara del mundo.
<code>Texture::loadFromFile</code> / <code>loadFromFileWithPixelPadding</code>	Cargar recursos gráficos.
<code>TextureAtlasPadding::get</code>	Elegir regiones concretas dentro de una textura atlas.

6. Funciones de audio y librerías externas

Función	Rol
<code>InitAudioDevice()</code>	Inicializa el subsistema de audio al comenzar.
<code>LoadSound(...)</code>	Carga el sonido del disparo.
<code>SetSoundVolume(...)</code>	Ajusta volumen del sonido.
<code>PlaySound(...)</code>	Reproduce disparos del jugador y enemigos.
<code>IsSoundPlaying(...)</code>	Evita superposición excesiva del mismo efecto.
<code>glfwInit()</code> , <code>glfwCreateWindow()</code> , <code>glfwMakeContextCurrent()</code>	Inicialización de ventana y contexto.
<code>glfwSetKeyCallback()</code> , <code>glfwSetMouseButtonCallback()</code> , etc.	Conectan los eventos del sistema con la lógica interna.
<code>gladLoadGLLoader(...)</code>	Carga funciones de OpenGL.
<code>glfwSwapBuffers()</code> , <code>glfwPollEvents()</code>	Presenta el frame y procesa eventos.

7. Secuencia lógica del juego en un frame

- 1 Se calcula deltaTime en el bucle principal de main.
- 2 Se llama a gameLogic(deltaTime).
- 3 Se actualiza viewport, limpieza de pantalla y métricas del renderer.
- 4 Se lee el input de teclado y se mueve al jugador.
- 5 La cámara sigue al jugador.
- 6 Se dibujan los fondos con parallax.
- 7 Se calcula la dirección del mouse para orientar la nave.
- 8 Si se hace click izquierdo, se crea una bala y se reproduce sonido.
- 9 Se actualizan balas, se chequean colisiones y se elimina lo que ya no sirve.
- 10 Se regenera vida o se reinicia la partida si el jugador muere.
- 11 Se generan y actualizan enemigos; si corresponde, disparan.
- 12 Se renderizan enemigos, nave, balas y barra de vida.
- 13 Se vacía el renderer y el frame termina.

8. Qué aprender de este video si estás estudiando C++

- Cómo separar plataforma, lógica del juego y renderizado.
- Cómo usar structs y métodos simples para modelar entidades.
- Cómo combinar std::vector con actualización por frame.
- Cómo usar deltaTime para movimiento independiente de FPS.
- Cómo encapsular input real de GLFW en una API propia más cómoda.
- Cómo aplicar álgebra vectorial básica para apuntado, rotación y seguimiento.
- Cómo cargar recursos y dibujar sprites con atlas.

9. Conclusión

El video muestra una implementación bastante clara de un shooter 2D pequeño. Las funciones más importantes no son solo las de C++ puro, sino la combinación entre funciones propias del gameplay, manejo de vectores, matemáticas con GLM, render con gl2d e input con GLFW. Si querés estudiar el proyecto, las funciones que más conviene mirar primero son **gameLogic**, **spawnEnemy**, **Enemy::update**, **Bullet::update** y **TiledRenderer::render**, porque concentran casi toda la lógica visible del juego.

Nota: este resumen fue armado a partir del video identificado y, sobre todo, del repositorio oficial enlazado por el autor.